



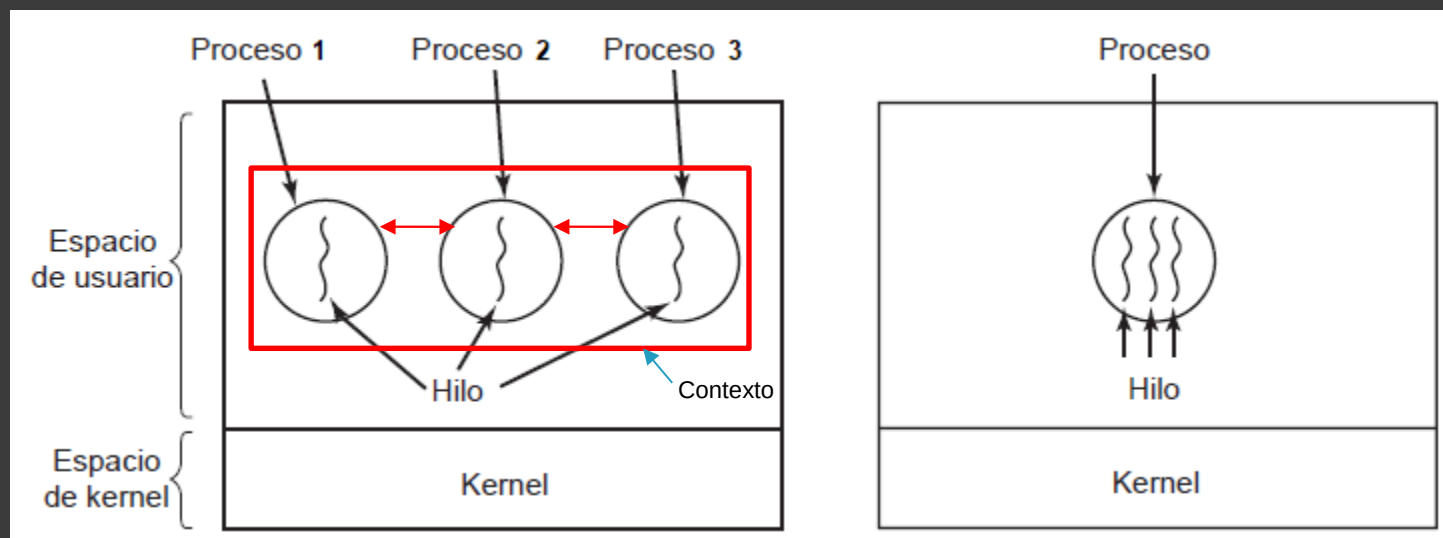
Centro Regional Universitario Córdoba IUA
Ing. en Informática

SISTEMAS EN TIEMPO REAL

Mag. Ing. José Ducloux
jducloux@iua.edu.ar

POSIX en C

- Hilos
- Procesos
- Sincronización de hilos
- **Sincronización de procesos**
 - **Comunicación entre procesos (IPC, Inter-Process Communication) usando técnicas de push o transmisión**



Técnicas de IPC de push o transmisión

- Los procesos **transmiten datos** entre sí para la comunicación.
- Esto implica una especie de **envío de mensajes o señalización** entre dos o más procesos.
- Los mensajes que se transmiten **no se almacenan en ningún lugar compartido** como un archivo o una memoria compartida, sino que son emitidos y recibidos por los procesos.
- Cada **mensaje** contiene una **serie de bytes** que se agrupan según una **interfaz, protocolo o estándar** bien definido.
- La **estructura de un mensaje** debe ser conocida por los procesos para establecer la comunicación.

Técnicas de IPC de pull o extracción

- También son técnicas IPC
 - Memoria compartida
 - Sistema de archivo

Diferencias entre IPC basadas en push y pull

- Medio de comunicación
 - Pull: recurso compartido o medio externo a ambos procesos y disponible en el espacio de usuario.
 - Push: requieren un canal de comunicación para enviar mensajes que no se almacenan en ningún intermediario.
 - Se tiene un buffer de mensajes temporal para cada proceso que contiene los mensajes entrantes. Este buffer de mensajes reside en el kernel y permanece mientras se ejecuta el proceso.

Diferencias entre IPC basadas en push y pull

- Acceso
 - Pull:
 - cada proceso extrae los mensajes disponibles del medio.
 - Push:
 - los mensajes se entregan desde un extremo transmisor a un extremo receptor.

Diferencias entre IPC basadas en push y pull

- Mecanismo de control
 - Pull:
 - el acceso concurrente al medio compartido debe controlarse para evitar que los datos se corrompan.
 - Push:
 - no requieren control.

Diferencias entre IPC basadas en push y pull

- Operación
 - Pull:
 - los procesos pueden operar de forma independiente ya que los mensajes se almacenan en el recurso compartido y recuperarse más tarde. Operación asíncrona.
 - Push:
 - ambos procesos deben estar en ejecución al mismo tiempo. Los mensajes se envían instantáneamente y se debe evitar la pérdida de mensajes entrantes. Operación síncrona.

Diferencias entre IPC basadas en push y pull

- Ambas técnicas de IPC deben usar un contenido que sea comprensible para los procesos.
- Esto significa que los procesos deben saber cómo crear y analizar los mensajes.
- Dado que los mensajes están formados por bytes, esto implica que ambos procesos deben saber cómo convertir un objeto en una serie de bytes, y cómo volver a formar el mismo objeto a través de los bytes recibidos.

Descriptores de archivo e IPC push

- Un descriptor de archivo puede representar un canal de IPC push para leer y escribir datos desde y hacia el canal representado.
- El primer paso para configurar un canal IPC push es definir una serie de descriptores de archivo.

Técnicas de IPC de push o transmisión de POSIX

- Comunicación en un solo host
 - Señales (signals): se pueden usar para notificar a un proceso sobre un evento (no crean un canal de comunicación entre los procesos).
 - Tuberías (pipes).
 - Colas de mensajes (message queues).
 - Sockets de dominio Unix.
- Comunicación en múltiples host
 - Sockets de red.

Señales

- Las señales se utilizan para notificar a un proceso o un hilo sobre la ocurrencia de un evento o un error.
- En los sistemas POSIX, los procesos y los hilos pueden enviar y recibir una serie de señales predefinidas.
- Una señal puede ser enviada por un proceso, un hilo o por el propio kernel.
- En su mayoría, las señales corresponden a eventos conocidos, pero hay dos señales POSIX que el usuario puede definir.
- Las señales se entregan de forma asíncrona (interrupción).
- Se debe incluir el archivo de cabecera “signal.h”

Señales

- No todas las señales son POSIX.
- GNU/Linux define también otras señales adicionales.

Signal	Standard	Action	Comment
SIGABRT	P1990	Core	Abort signal from abort(3)
SIGALRM	P1990	Term	Timer signal from alarm(2)
SIGBUS	P2001	Core	Bus error (bad memory access)
SIGCHLD	P1990	Ign	Child stopped or terminated
SIGCLD	-	Ign	A synonym for SIGCHLD
SIGCONT	P1990	Cont	Continue if stopped
SIGEMT	-	Term	Emulator trap
SIGFPE	P1990	Core	Floating-point exception
SIGHUP	P1990	Term	Hangup detected on controlling terminal or death of controlling process
SIGILL	P1990	Core	Illegal Instruction
SIGINFO	-		A synonym for SIGPWR
SIGINT	P1990	Term	Interrupt from keyboard
SIGIO	-	Term	I/O now possible (4.2BSD)
SIGIOT	-	Core	IOT trap. A synonym for SIGABRT
SIGKILL	P1990	Term	Kill signal
SIGLOST	-	Term	File lock lost (unused)
SIGPIPE	P1990	Term	Broken pipe: write to pipe with no readers; see pipe(7)
SIGPOLL	P2001	Term	Pollable event (Sys V); synonym for SIGIO
SIGPROF	P2001	Term	Profiling timer expired
SIGPWR	-	Term	Power failure (System V)
SIGQUIT	P1990	Core	Quit from keyboard
SIGSEGV	P1990	Core	Invalid memory reference
SIGSTKFLT	-	Term	Stack fault on coprocessor (unused)
SIGSTOP	P1990	Stop	Stop process
SIGTSTP	P1990	Stop	Stop typed at terminal
SIGSYS	P2001	Core	Bad system call (SVr4); see also seccomp(2)
SIGTERM	P1990	Term	Termination signal
SIGTRAP	P2001	Core	Trace/breakpoint trap
SIGTTIN	P1990	Stop	Terminal input for background process
SIGTTOU	P1990	Stop	Terminal output for background process
SIGUNUSED	-	Core	Synonymous with SIGSYS
SIGURG	P2001	Ign	Urgent condition on socket (4.2BSD)
SIGUSR1	P1990	Term	User-defined signal 1
SIGUSR2	P1990	Term	User-defined signal 2
SIGVTALRM	P2001	Term	Virtual alarm clock (4.2BSD)
SIGXCPU	P2001	Core	CPU time limit exceeded (4.2BSD); see setrlimit(2)
SIGXFSZ	P2001	Core	File size limit exceeded (4.2BSD); see setrlimit(2)
SIGWINCH	-	Ign	Window resize signal (4.3BSD, Sun)

Señales

- Ejemplo 1
 - Se utiliza la función `signal()` para asignar diferentes controladores de señales específicas.
 - Se tienen los controladores de:
 - Señales definidas por el usuario.
 - `SIGINT` (ctrl+c).
 - `SIGKILL`.
 - Las señales se envían desde la terminal al proceso en ejecución en segundo plano usando los comandos:
 - `kill -SIGUSR2 'PID'`
 - `kill -SIGUSR1 'PID'`
 - `kill -SIGINT 'PID'`
 - `kill -SIGKILL 'PID'`

Señales

- Un proceso también puede enviar una señal a otro proceso si conoce su PID.
- La función `kill()` se utiliza para enviar señales (hace exactamente lo mismo que su versión de línea de comandos).
 - Acepta dos parámetros: el primero es el PID del destino y el segundo es el número de señal.
- También es posible que un proceso o un hilo use las funciones `kill()` para enviarse una señal a sí mismo.

Tuberías (pipes)

- Son canales unidireccionales entre dos procesos que se pueden usar para intercambiar datos.
- El kernel retiene en un buffer los datos escritos hasta que se leen.
- Para comunicación bidireccional, se deben crear dos pipes.
- Cuando se crea una pipe se obtienen dos descriptores de archivo:
 - Un descriptor se usa para leer desde la pipe.
 - Un descriptor se usa para escribir en la pipe.
- La pipe se crea con la función `pipe()`, la cual acepta y abre los dos descriptores de archivo.
- Para leer y escribir la pipe, se usan las funciones `read()` y `write()` relacionadas con descriptores de archivo.
- Los datos se manejan en unidades de bytes.

Tuberías (pipes)

- Ejemplo 2
 - Se utiliza `fork()` para crear dos procesos.
 - Se llama a la función `pipe()`, la cual acepta un arreglo y abre los dos descriptores de archivo.
 - En `fds[0]`: descriptor de lectura de la pipe.
 - En `fds[1]`: descriptor de escritura de la pipe.
 - La pipe se crea y se abre antes de llamar a `fork()` por lo que estará disponible para el proceso padre y para el proceso hijo. Los descriptores de archivo estarán abiertos y disponibles para ambos procesos.
 - El padre lee la pipe y el hijo escribe en la pipe. El padre cierra el descriptor de escritura; el hijo cierra el descriptor de lectura.
 - Para leer la pipe se usa la función `read()`.
 - Para escribir sobre la pipe se usa la función `write()`.

Colas de mensajes

- Son canales de comunicación bidireccionales.
- Los elementos de información son mensajes.
- Las colas de mensajes se alojan en el kernel. Se utiliza un descriptor de archivo para manejar la cola.
- Se especifica el tamaño de los mensajes y la cantidad de mensajes en la cola.
- Las funciones relacionadas con colas se encuentran en el archivo de cabecera “mqqueue.h”.
 - mq_open();
 - mq_send();
 - mq_receive();
 - mq_close();
 - mq_unlink();

Colas de mensajes

- Ejemplo 3
 - Es el ejemplo 2 pero se utilizan colas de mensajes para la comunicación.
 - Debe enlazarse con la biblioteca rt.
 - La salida del programa es exactamente la misma que la del ejemplo 2.
 - Se establecen atributos de la cola como el tamaño de cada mensaje y la cantidad máxima de mensajes.
 - El proceso padre crea la cola de mensajes y luego lee la cola.
 - El proceso hijo escribe en la cola de mensajes.

Programación de sockets de POSIX

- Permiten conectar dos o más procesos que residen en un mismo host o en hosts diferentes que tienen una conectividad entre ellos.
- La programación de sockets ocurre en la capa de transporte y es por eso que principalmente se debe tener conocimiento de los diferentes protocolos.
- Biblioteca de sockets POSIX
 - Sockets de dominio Unix.
 - Sockets de red.



Centro Regional Universitario Córdoba IUA
Ing. en Informática

¿Preguntas?